Data Structures and Algorithms - Semester 1 Assignments

Weekly Hands-On Programming Tasks • Modules 1 to 10

Assignment 1: Asymptotic Complexity Analysis

Week 1 **Theoretical & Analytical**

Objective: Analyze algorithm growth rates and compute runtime behavior.

- **Task 1:** Determine the worst-case time complexity using Big-O notation for three nested iterative algorithms.
- **Task 2:** Prove mathematically why an $O(N \log N)$ sorting algorithm outperforms an $O(N^2)$ algorithm for $N = 10^6$.

Assignment 2: Dynamic Array Operations & Matrix Manipulation

Week 2 **Programming**

Objective: Implement standard array manipulations and 2D matrix transformations.

- **Task 1:** Write a program to rotate an $N \times N$ integer matrix by 90 degrees clockwise in-place.
- **Task 2:** Implement a function to find the missing number in an array containing numbers from 1 to N with $O(N)$ time and $O(1)$ space.

Assignment 3: Pointer Arithmetic and Heap Allocation

Week 3 **Memory Management**

Objective: Manage dynamic memory allocation safely using pointers.

- **Task 1:** Write a C program that dynamically allocates a 1D array using `malloc()`, resizes it dynamically using `realloc()`, and ensures no memory leaks occur using `free()`.
- **Task 2:** Implement a safe string reversal function utilizing raw pointer arithmetic without using array index notation.

Assignment 4: Singly Linked List Implementation

Week 4 **Data Structures**

Objective: Build a complete Singly Linked List from scratch.

- **Task 1:** Implement functions for insertion at the head, tail, and specific indexed positions.
- **Task 2:** Write an algorithm to reverse a Singly Linked List iteratively and recursively.

Assignment 5: Doubly & Circular Linked Lists

Week 5 **Advanced Pointers**

Objective: Handle multi-directional and looped pointer chains.

- **Task 1:** Implement a Doubly Linked List with bidirectional traversal and node deletion capabilities.
- **Task 2:** Detect whether a given linked list contains a cycle using Floyd's Tortoise and Hare algorithm.

Assignment 6: Stack Applications

Week 6 **LIFO Architecture**

Objective: Solve expression parsing problems using Stack data structures.

- **Task 1:** Implement a stack using dynamic arrays.
- **Task 2:** Write a program to convert an Infix mathematical expression to Postfix notation and evaluate valid postfix strings.

Assignment 7: Queue Implementations and Variants

Week 7 **FIFO Architecture**

Objective: Optimize queue storage using circular configurations.

- **Task 1:** Implement a Circular Queue using arrays to prevent linear space wastage.
- **Task 2:** Simulate a CPU Round-Robin job scheduler using a Queue data structure.

Assignment 8: Elementary Sorting Techniques

Week 8 **Sorting Algorithms**

Objective: Analyze and code basic quadratic sorting algorithms.

- **Task 1:** Implement Bubble Sort, Selection Sort, and Insertion Sort.
- **Task 2:** Count and compare the total number of element comparisons and swaps made by each sorting algorithm on a random array of size 1000.

Assignment 9: Recursive Problem Solving

Week 9 **Recursion**

Objective: Break complex tasks down using self-referential functions.

- **Task 1:** Write a recursive function to solve the classic Tower of Hanoi puzzle for N disks.
- **Task 2:** Implement recursive binary search on a sorted array.

Assignment 10: Divide and Conquer Sorting

Week 10 Advanced Algorithms

Objective: Implement high-performance sorting algorithms.

- **Task 1:** Code Merge Sort using the divide-and-conquer paradigm.
- **Task 2:** Code Quick Sort using Lomuto or Hoare partition schemes and analyze pivot selection impact.